

Missing Production Migrations

Project: TSCopier · **Environment:** Production `sxkpcovbyaficvtpksdo` · **Date checked:** 2026-08-16

VERIFIED MISSING VIA DIRECT DB CHECKS

Summary

Production is **missing 4 database changes** that staging already has.

Two of them are a **live problem**: the app code is already running on production, and it writes to a place in the database that was never created. Right now, when a user clicks "Report" on a trade (or asks the assistant to report a trade), the action fails with an error.

The other two are **prep work for a feature that is switched off**. They are safe to wait.

#	CHANGE	ON PROD?	PRIORITY	WHY
1	<code>20260810000000_trade_reports</code>	MISSING	HIGH	The table where trade reports are stored — live code writes to it
2	<code>20260816000000_trade_reports_signal_id</code>	MISSING	HIGH	Adds a link from a report back to its signal; needs #1 first
3	<code>20260730120000_layering_modes_foundation</code>	MISSING	MEDIUM	Extra storage space for a future (off) feature
4	<code>20260731120000_layering_plans</code>	MISSING	MEDIUM	The future feature's main storage; needs #3 first

Already on prod: `enforce_plan_broker_channel_limits` (registered as `20260805082647`), verified working since 2026-08-05.

What each change does

1. Trade reports table **HIGH — LIVE PROBLEM**

What it does. Creates the `trade_reports` table — the place where user complaints about trades ("wrong entry", "wrong stop-loss", "not executed", etc.) are stored for support staff to review. Each report keeps a snapshot of the trade (symbol, direction, ticket, broker, entry price, stop-loss, take-profit, lot size), plus a category, a reason, and a status (`open` or `resolved`). Users can only see and file their **own** reports.

Where it's used in the code

- **Manual report button:** `src/components/trades/ReportTradeModal.tsx` — saves the report from the trade detail modal.
- **Assistant report tool:** `supabase/functions/assistant-chat/index.ts` — the assistant's `report_trade` tool saves a report the same way.

What happens without it. Every attempt to file a report fails with `ERROR 42P01: relation "trade_reports" does not exist`. This is a real, user-facing defect on prod.

2. Link reports to their signal HIGH

What it does. Adds a `signal_id` column to each trade report, so a report can point back to the signal it came from. This matters for **skipped or non-actionable** trades — the ones with no symbol or ticket — because the assistant can now file reports on those too, and support needs to know which signal the report was about. The column is optional and old reports are untouched.

Where it's used in the code

- **Writer:** `supabase/functions/assistant-chat/index.ts` (`report_trade`) — saves the `signal_id` when the assistant files a report, so it can be traced back to the original signal.
- Nothing else reads this column; it exists for support traceability.

What happens without it. Once change #1 is applied, reports still save — they just can't be linked back to their signal.

3. Storage space for future layering modes MEDIUM — FEATURE OFF

What it does. Adds two empty boxes to the table that holds the ladder of pending orders for range trades: `layer_plan_id` (which plan a waiting order belongs to) and `layer_plan_metadata` (extra details about that link). It also adds an index so lookups are fast. Existing orders are untouched and behave exactly as before.

Where it's used in the code

- **Writers:** `worker/src/manualPlanning/layeringPlanPersistence.ts` fills these boxes on the waiting orders; `worker/src/layeringPlanLifecycle.ts` and `worker/src/tradeExecutor/layeringModeBrokerPending.ts` read them as the orders move through execution.

What happens without it. The worker code already writes and reads these boxes. On prod the boxes don't exist, so those writes fail **the moment** the feature is switched on. While the feature stays off, nothing breaks.

4. Main storage for the future layering feature MEDIUM — FEATURE OFF

What it does. Creates the `layering_plans` table — the saved plan for a range-trade ladder (which mode, its lifecycle status, the frozen calculator details, and a fingerprint to spot duplicate plans). It also adds the `activate_layering_plan` function that turns a saved plan into real waiting orders, plus a set

of empty boxes on the waiting orders for tracking each order through the broker (submitted, waiting, confirmed, cancelled). Internal-only – users have no access to this table.

Needs #3 first: its index on `range_pending_legs(layer_plan_id, step_idx)` requires the `layer_plan_id` column added in change #3.

Where it's used in the code

- **Writer:** `worker/src/manualPlanning/layeringPlanPersistence.ts` – saves the plan and calls `activate_layering_plan(...)` to turn it into waiting orders.
- **Readers:** `worker/src/tradeExecutor/layeringModeBrokerPending.ts` (submits orders and records their broker status), `layeringModeBrokerPendingRecovery.ts` (reads the saved plan to resume after a restart), `layeringPlanLifecycle.ts`, and `worker/src/virtualPendingMonitor.ts`.

What happens without it. With no table on prod, saving a plan fails; with no status boxes, tracking submitted orders fails; and if the worker restarts mid-trade, there is no saved plan to read, so it can't continue. Pure additive – nothing runs this feature until it's switched on, and it's behind kill-switch flags (`LAYERING_MODES_EXECUTION_ENABLED` , `LAYERING_MODES_PREPARE_ONLY`) that only allowlist staging accounts can use.

How we know this

We ran the same 5 checks against **staging** (which has the changes) and **prod**:

CHECK	STAGING (HAS IT)	PROD
Does the <code>trade_reports</code> table exist?	Yes	No
Any tables matching "%report%"?	Yes	None
Any tables matching "%trade_report%"?	Yes	None
Changes registered in the migration list?	Both versions	None
Can we count the rows in <code>trade_reports</code> ?	Yes (8 rows)	Error: table does not exist

Prod also fails the checks for the layering changes: no `layering_plans` table, no `layer_plan_id` / `layer_plan_metadata` / `broker_client_reference` columns on the waiting orders, and no layering guard triggers.

Because staging passes the same checks, we know the checking method is sound – the prod failures are real.

What to do

Recommended: apply changes #1 and #2 to prod now – they fix the live trade-report failure for both the manual button and the assistant. Apply them the usual way, through the **main Supabase Dashboard**

(SQL Editor), because the dashboard does **not** auto-register migrations — the tracker entry has to be added by hand afterwards.

1. In the Supabase Dashboard, open the **prod** project (`sxkpcovbyaficvtpkpsdo`) → **SQL Editor**.
2. Paste and run `supabase/migrations/20260810000000_trade_reports.sql`.
3. Then paste and run `supabase/migrations/20260816000000_trade_reports_signal_id.sql` — this order matters, change #2 depends on #1.
4. Register both in the migration tracker so future scripts see them as applied:

```
INSERT INTO supabase_migrations.schema_migrations (version, statements, name)
VALUES
  ('20260810000000', ARRAY['-- applied via dashboard'], '20260810000000_trade_reports.sql'),
  ('20260816000000', ARRAY['-- applied via dashboard'],
  '20260816000000_trade_reports_signal_id.sql')
ON CONFLICT (version) DO NOTHING;
```

1. Re-check that the table, column, and indexes exist (see "How we know this").

Alternative (scripted): `scripts/apply-missing.sh` diffs the local migration files against the tracker and applies + registers every missing one via the Management API. Point its `PROJECT` variable at the prod ref first — it defaults to the staging project.

Decision needed: whether to also apply #3 and #4 to prod now, or wait until the layering feature is scheduled. They're safe either way — they don't change anything until the feature is switched on.

Also pending (code, not a database change): redeploy the `assistant-chat` edge function to staging then prod with the skipped-trade `report_trade` fix (the code is committed; the function is deployed manually).

Status update — 2026-08-17

- All four migration files (incl. #1 and #2) are now **committed to the repo** on the `staging` and `dev` branches and pushed to `origin` + `upstream`, so they can no longer be lost.
- The `assistant-chat` edge function change (skipped / non-actionable `report_trade` with `signal_id`, plus the "last trade" few-shot guidance) and the clickable assistant trade-detail card (opens the copier-log detail modal) are on the same branches.
- **Still required — apply + register #1 and #2 on prod (`sxkpcovbyaficvtpkpsdo`) before deploying the updated `assistant-chat` function.** Then deploy the function to staging → prod. If the column does not exist when the function ships, every `report_trade` insert fails with `column "signal_id" does not exist`.
- Re-check prod afterwards: `to_regclass('public.trade_reports')` should return `trade_reports` and `trade_reports.signal_id` should exist.

Plain English: TSCopier's production database is missing 4 things. Two of them (the trade-reports table) mean users can't file a trade complaint right now – the button errors out. The other two are behind-the-scenes prep for a future feature that isn't turned on yet. Staging has all 4.